

Let us add the first Swarm agent node (worker node) as shown in Figure 7.

```
root@swarm-agent1:~# docker swarm join 10.140.0.5:2377
This node joined a Swarm as a worker.
root@swarm-agent1:~#
```

Figure 7: Adding a worker node

Let's go back to the Swarm master node to see the latest Swarm Mode status.

Similarly, we can add other Swarm agent nodes to the swarm cluster and see the listings, as shown in Figure 8.

```
root@swarm-master1:~# docker node ls
ID                NAME                MEMBERSHIP  STATUS  AVAILABILITY  PA
-----
1jvba096dc7v8znpmd6vqg  swarm-agent1  Accepted    Ready   Active         Ia
dlu1603hndw130maiqriv  swarm-master1 Accepted    Ready   Active         Ia
oc93w1wv44n1a1etqf    swarm-agent2  Accepted    Ready   Active         Ia
root@swarm-master1:~#
```

Figure 8: Listing the three-node cluster

Creating services and tasks

Let's try creating a single service called *collab*, which uses the *busybox* image from Dockerhub, and all it does is ping the *collabnix.com* website.

```
root@swarm-master1:~# docker service create --name collab busybox ping collabnix.com
21ak3de161ed4d0w0j1lgyy
root@swarm-master1:~# docker service ls
ID                NAME                REPLICAS  IMAGE     COMMAND
-----
21ak3de161ed4d0w0j1lgyy collab 1/1 busybox ping collabnix.com
root@swarm-master1:~#
```

Figure 9: Creating a Docker service

```
root@swarm-master1:~# docker service tasks collab
ID STATE MODE                NAME                SERVICE  IMAGE     LAST STATE  DESIRE
-----
5f1j0c4m16k6urgd96f132ny collab.1 collab busybox Running 2 minutes Running
g swarm-master1
root@swarm-master1:~#
```

Figure 10: Listing the Docker service

A task is an atomic unit of service. We actually create a task whenever we add a new service. For example, as shown in Figure 11, we have created a task called *collab*.

```
root@swarm-master1:~# docker service scale collab=5
collab scaled to 5
```

Figure 11: Scaling the service

Scaling the service

To scale the service to 5, run the command shown in Figure 12.

```
root@swarm-master1:~# docker service tasks collab
ID STATE MODE                NAME                SERVICE  IMAGE     LAST STATE  DESIRE
-----
5f1j0c4m16k6urgd96f132ny collab.1 collab busybox Running 6 minutes Running
g swarm-master1
5f1j0c4m16k6urgd96f132ny collab.2 collab busybox Running 42 seconds Running
g swarm-agent-3
5f1j0c4m16k6urgd96f132ny collab.3 collab busybox Running 42 seconds Running
g swarm-agent-3
5f1j0c4m16k6urgd96f132ny collab.4 collab busybox Running 42 seconds Running
g swarm-agent-3
5f1j0c4m16k6urgd96f132ny collab.5 collab busybox Running 42 seconds Running
g swarm-agent-1
root@swarm-master1:~#
```

Figure 12: Listing the Docker containers across the cluster

Now you can see that there are five different containers running across the cluster, as shown in Figure 13.

The command in Figure 13 declares a desired state on your swarm of five *busybox* containers, reachable as a single, internally load balanced service of any node in your swarm.

```
root@swarm-master1:~# docker ps
CONTAINER ID        IMAGE               COMMAND
-----
21ak3de161ed4d0w0j1lgyy busybox             ping collabnix.com
5f1j0c4m16k6urgd96f132ny busybox             ping collabnix.com
5f1j0c4m16k6urgd96f132ny busybox             ping collabnix.com
5f1j0c4m16k6urgd96f132ny busybox             ping collabnix.com
5f1j0c4m16k6urgd96f132ny busybox             ping collabnix.com
root@swarm-master1:~#
```

Figure 13: Demonstrating rolling updates in Swarm Mode

Rolling updates

Updating a service is pretty simple. The 'Docker service update' is feature-rich and provides loads of options to play around with the service. Let's try updating the *redis* container from 3.0.6 to 3.0.7 with a 10s delay and parallelism count of 2.

Promoting a node to the manager group

Let's try to promote Swarm Agent Node-1 to the manager group, as shown in Figure 14.

```
root@swarm-master1:~# docker node ls
ID                NAME                MEMBERSHIP  STATUS  AVAILABILITY  MANAGER STATUS
-----
21ak3de161ed4d0w0j1lgyy swarm-agent1  Accepted    Ready   Active
5f1j0c4m16k6urgd96f132ny swarm-master1 Accepted    Ready   Active   Leader
oc93w1wv44n1a1etqf    swarm-agent2  Accepted    Ready   Active
root@swarm-master1:~# docker node promote 5f1j0c4m16k6urgd96f132ny
Promoting node 5f1j0c4m16k6urgd96f132ny to manager.
root@swarm-master1:~# docker node ls
ID                NAME                MEMBERSHIP  STATUS  AVAILABILITY  MANAGER STATUS
-----
21ak3de161ed4d0w0j1lgyy swarm-agent1  Accepted    Ready   Active
5f1j0c4m16k6urgd96f132ny swarm-agent3  Accepted    Ready   Active
oc93w1wv44n1a1etqf    swarm-agent2  Accepted    Ready   Active   Leader
root@swarm-master1:~#
```

Figure 14: Promoting worker node to the manager group

To summarise, Docker comes with a distribution platform, and makes multi-host and multi-container orchestration easy. It has new API objects like services and nodes that will let you use the Docker API to deploy and manage apps on a group of Docker engines, and provides scaling, promotion and rolling updates for the cluster nodes. 

References

- 1) <https://blog.docker.com/2016/06/docker-1-12-built-in-orchestration>
- 2) <http://www.collabnix.com>
- 3) <https://github.com/docker>

By: Ajeet S. Raina

The author is a project lead engineer with Dell India R&D. He is a 'Docker Captain' (honoured by Docker Inc.). He blogs about the latest Docker releases through his personal space www.collabnix.com. You can reach him at Ajeet_Raina@dell.com.